Course code: CSA1013 Name: S Jefna Princy

Course: Software Engineering Reg No.: 192572058

CO 3 Assessment Tool – 1

AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System

GIT & DOCKER TECHNICAL ASSIGNMENT

1. Problem Analysis

1.1 Problem Statement

Urban areas experience increasing noise pollution due to transportation, industries, construction activities, and other human activities. Traditional noise monitoring methods depend mainly on periodic manual measurements, which cannot continuously capture changes in noise levels.

The proposed **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** uses IoT noise sensors to collect real-time noise data. The collected data is stored in the cloud and analyzed using Artificial Intelligence and Machine Learning techniques. GIS mapping is used to identify noise hotspots, while an alert mechanism notifies authorities when noise levels exceed permissible limits.

1.2 Project Objectives

The main objectives are:

1. To continuously monitor urban noise levels.
2. To collect real-time data from noise sensors.
3. To identify high-noise areas using GIS.
4. To analyze and predict noise patterns using AI.
5. To generate real-time alerts for excessive noise.
6. To identify major sources of noise pollution.
7. To generate environmental compliance reports.
8. To support better urban planning and public health protection.

1.3 Functional Requirements

The system shall:

- Collect noise-level data from IoT sensors.
- Store sensor data in a centralized database.
- Process and analyze noise data.
- Detect abnormal or excessive noise levels.
- Predict future noise trends using AI.
- Display noise levels using GIS maps.
- Identify noise pollution hotspots.
- Generate alerts when limits are exceeded.
- Generate reports for authorities.
- Provide monitoring information through a dashboard.

1.4 Non-Functional Requirements

Requirement	Description
Performance	The system should process sensor data with minimum delay.
Scalability	It should support additional sensors and users.
Security	Sensor and user data should be protected from unauthorized access.
Reliability	The system should continue functioning even when temporary failures occur.
Availability	Monitoring services should be available continuously.
Maintainability	The software should be modular and easy to update.
Usability	The dashboard should be simple and easy to understand.

1.5 Expected Outcomes

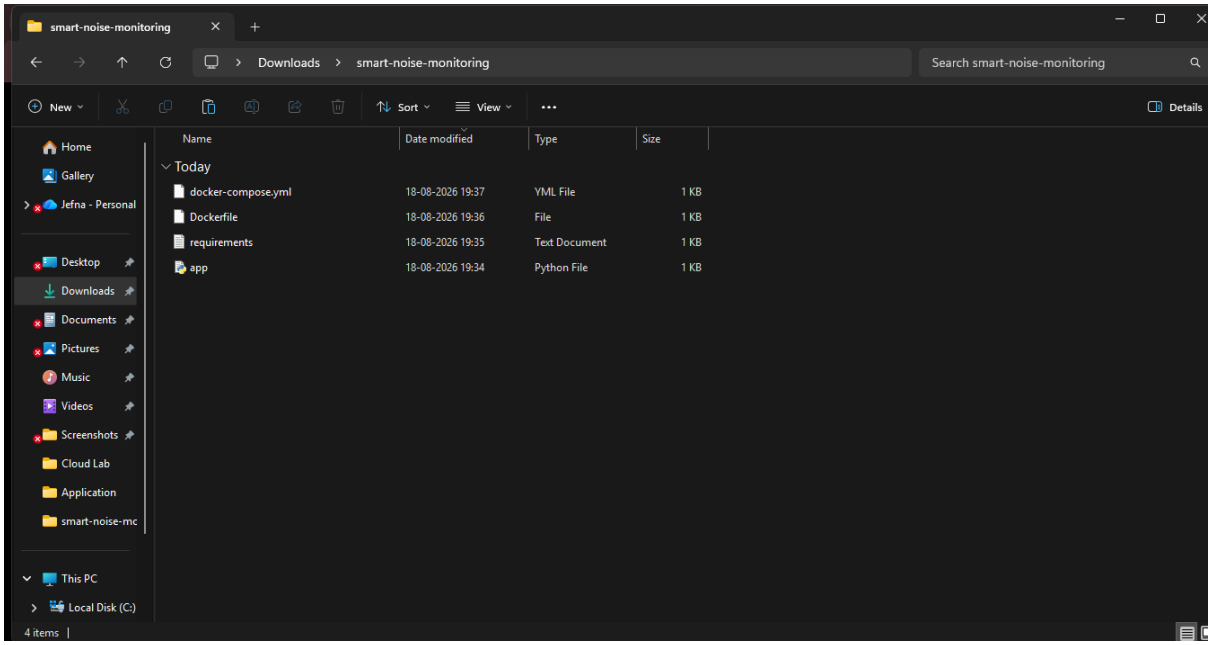
The proposed system is expected to provide:

- Real-time noise monitoring.
- Early identification of noise hotspots.
- Better environmental monitoring.
- Faster response to excessive noise.
- Improved urban planning.
- Better public health protection.
- Increased regulatory compliance.
- Improved quality of urban living.

2. Project Structure Design

The project repository is organized into separate folders so that different components can be developed and maintained easily.

Repository Structure



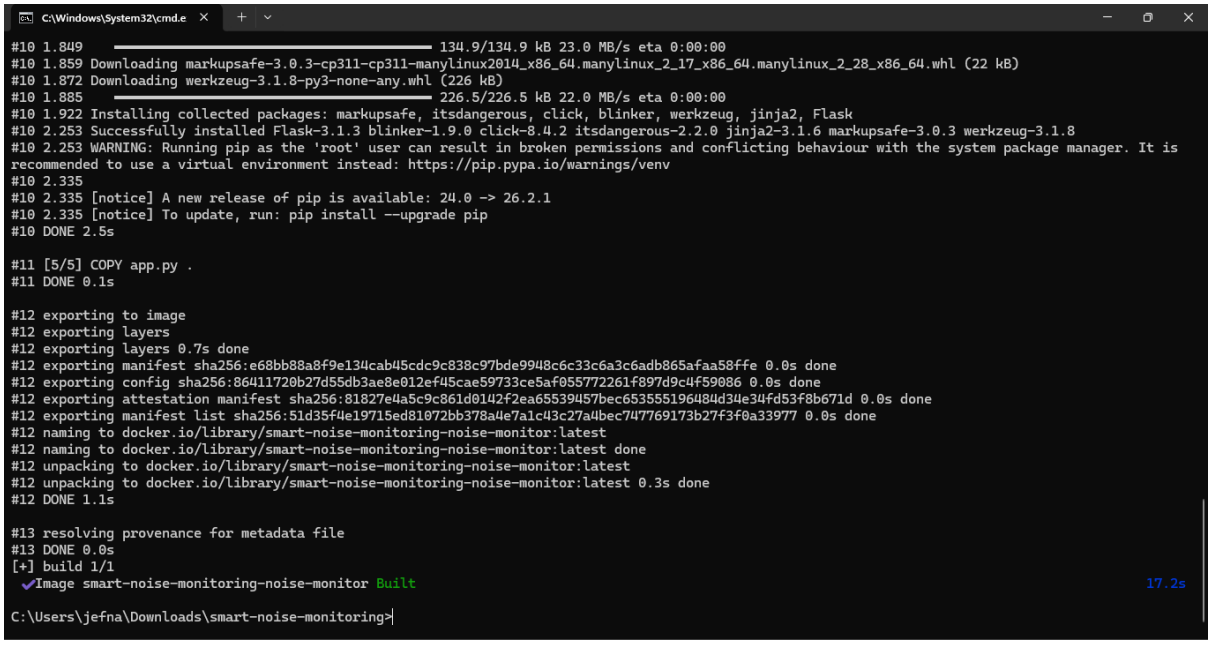
Purpose of Major Folders

- **app/** – Contains the main application and API code.
- **ai/** – Contains AI/ML prediction and analysis modules.
- **data/** – Stores sample or input noise data.

- **dashboard/** – Contains the monitoring dashboard interface.
- **tests/** – Contains testing files.
- **Dockerfile** – Defines how the application is containerized.
- **docker-compose.yml** – Defines and manages multiple services.
- **requirements.txt** – Lists the required Python libraries.
- **README.md** – Provides project information and instructions.
- **.gitignore** – Prevents unnecessary files from being uploaded to Git.

3. Git Repository Initialization

- Git is used to maintain the project source code and track changes



Steps:

mkdir smart-noise-monitoring

cd smart-noise-monitoring

git init

Then the project files are added:

git add .

The first commit is created:

git commit -m "Initial project setup"

A remote repository can then be connected:

git remote add origin <repository-url>

git push -u origin main

Important Git Files

README.md
Contains project description, setup instructions and usage information.

.gitignore
Prevents unnecessary files such as cache files, environment files and temporary files from being committed.

requirements.txt
Contains the dependencies required to run the application.

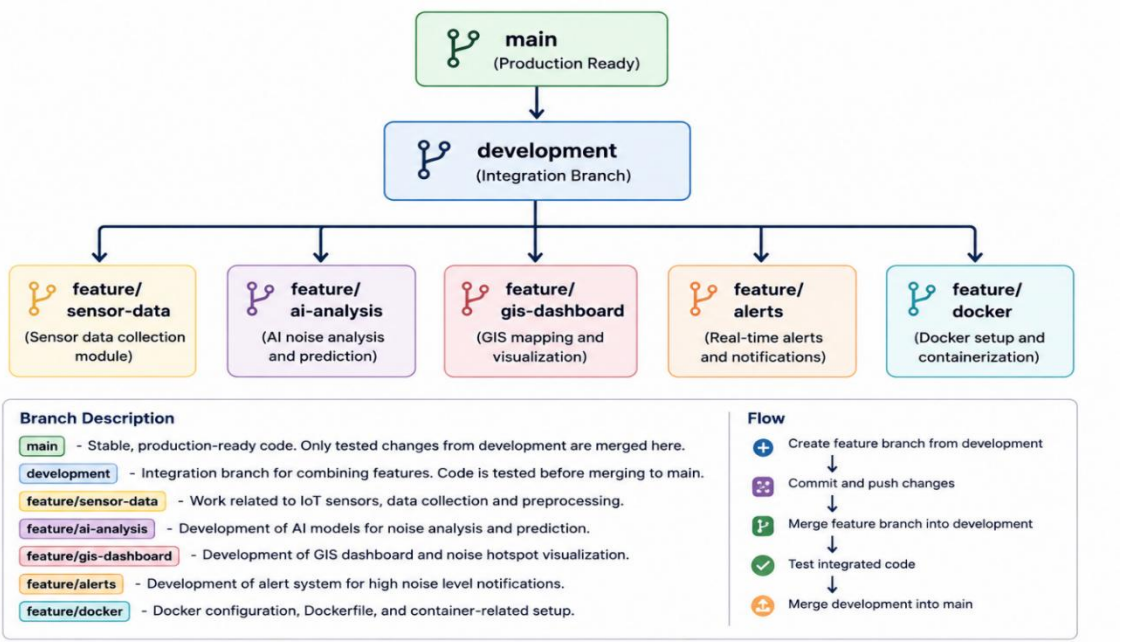
4. Git Branching Strategy

For this project, a **feature-based branching strategy** is suitable because different components can be developed independently.

Branch Structure

Explanation

- **main** – Contains the stable version of the project.
- **development** – Used for integrating new features.
- **feature/sensor-data** – Used for sensor data collection.



- **feature/ai-analysis** – Used for AI-based analysis.
- **feature/gis-dashboard** – Used for GIS visualization.
- **feature/alerts** – Used for the alert system.
- **feature/docker** – Used for containerization.

Example

git checkout -b development

git checkout -b feature/ai-analysis

After completing a feature:

git add .

git commit -m "Add AI noise prediction module"

The feature can then be merged into the development branch after testing.

Why This Strategy?

This strategy reduces conflicts because each feature can be developed separately. It also keeps the **main branch stable** and makes collaboration easier.

5. Version Control Workflow

The Git workflow followed in the project is:

Important Git Operations

Check project status

git status

Add changes

git add .

Commit changes

git commit -m "Add noise monitoring API"

Push changes

git push

Get latest changes

git pull

Create a branch

git checkout -b feature/gis-dashboard

Meaningful Commit Messages

Examples:

Initial project setup

Add sensor data collection module

Add noise prediction model

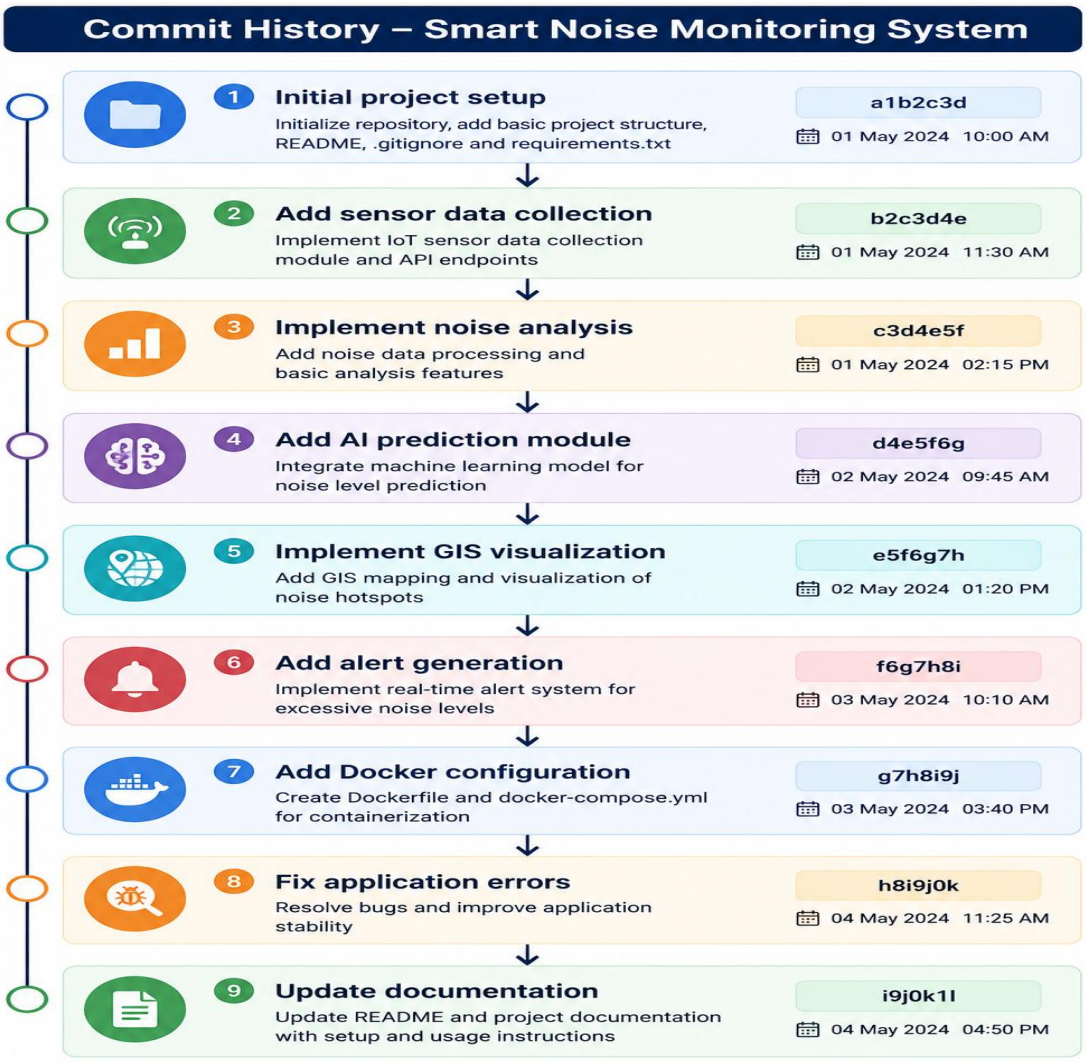
Implement GIS dashboard

Add excessive noise alert system

Add Docker configuration

Fix API validation

Update project documentation



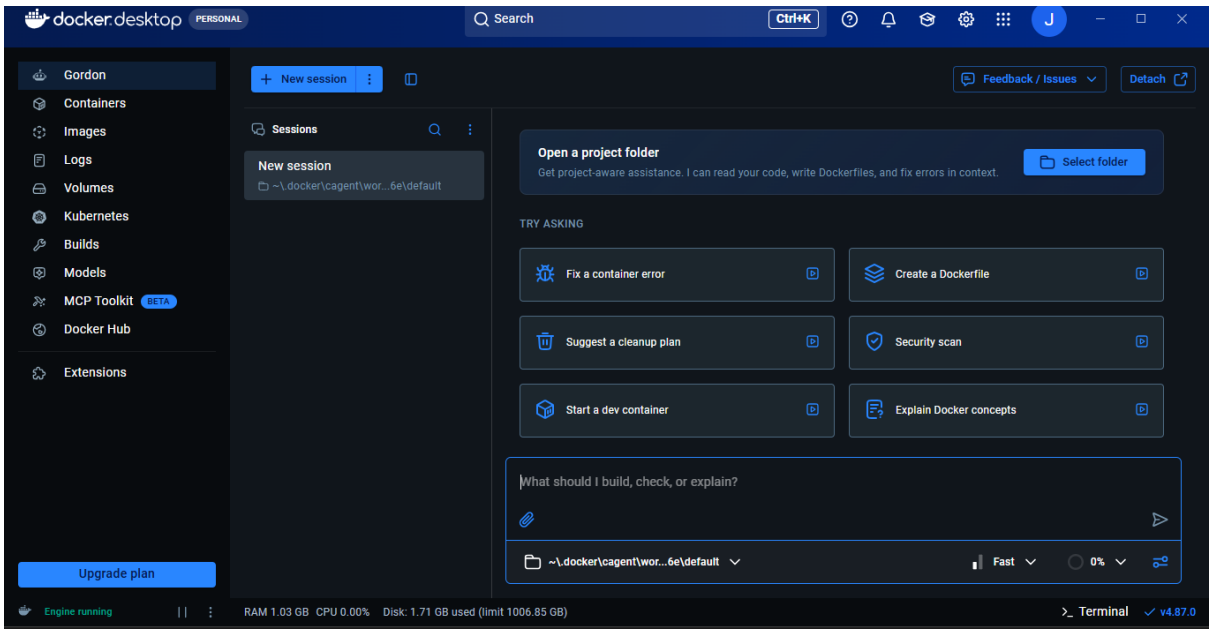
Meaningful commit messages make it easy to understand the history of the project.

The following command can be used to display the history:

git log –online

Analysis

Each commit represents a meaningful development activity. The commits are organized in chronological order and describe the changes clearly. This makes the project easier to maintain, debug and review.



7. Docker Environment Design

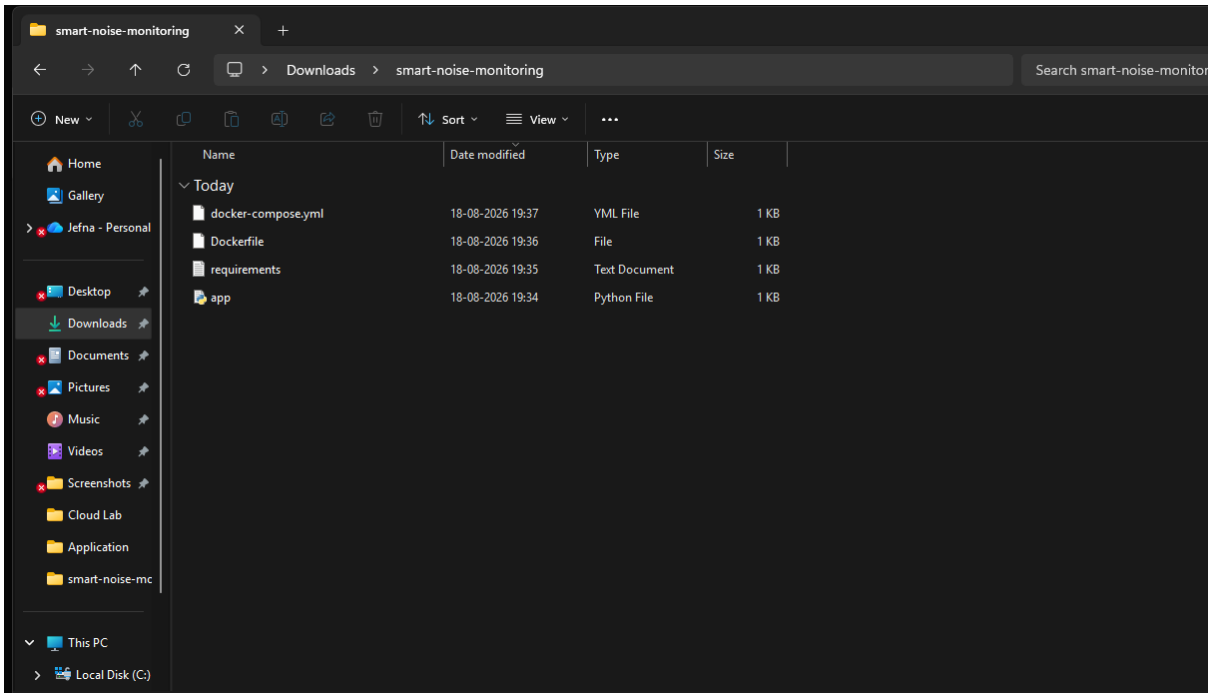
Docker is used to package the application and its dependencies into a portable container.

Why Docker?

Docker is suitable because:

- It provides a consistent environment.
- It avoids dependency conflicts.
- It makes deployment easier.
- It improves portability.
- It allows different services to run independently.
- It simplifies application testing.

Components to Containerize



The proposed system can contain:

Noise Monitoring Application

↓

AI Analysis Service

↓

Database

↓

Dashboard

For a simple prototype, the application and database can be containerized using Docker Compose.

8. Docker file Development

Create a file named:

Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "app/main.py"]

9. Docker Compose Design

Create:

docker-compose.yml

Docker Compose

version: "3.8"

services:

app:

build: .

container_name: noise-monitoring-app

ports:

- "8000:8000"

depends_on:

- database

database:

image: postgres:15

container_name: noise-monitoring-db

environment:

POSTGRES_DB: noise_db

POSTGRES_USER: noise_user

POSTGRES_PASSWORD: noise_password

ports:

- "5432:5432"

```
  volumes:
    - noise_data:/var/lib/postgresql/data
volumes:
  noise_data:
```

Services

Application Service

Runs the main noise monitoring application.

Database Service

Stores sensor readings, analysis results and other system information.

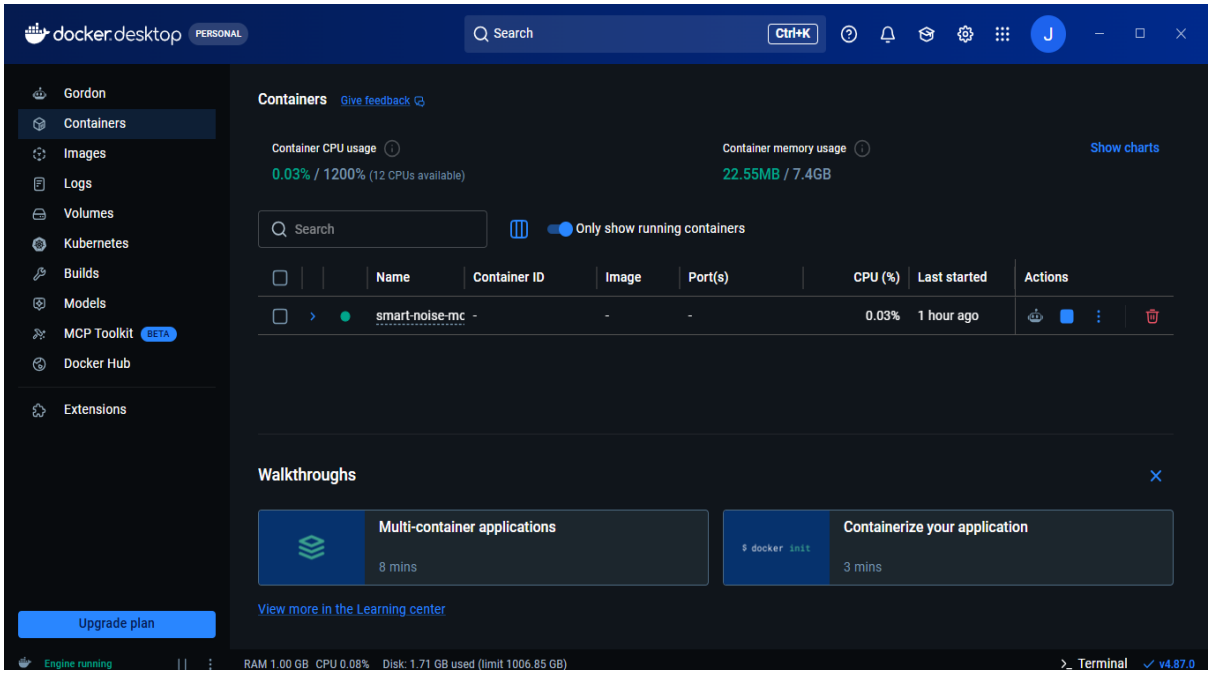
Network

Docker Compose automatically creates a network so the application can communicate with the database.

Volume

The database volume ensures that stored data is not lost when the container is restarted.

10. Container Deployment and Testing



First build and start the containers: docker compose up –build

To run them in the background: docker compose up -d

Check running containers: docker ps

View application logs: docker compose logs

Stop the containers: docker compose down

Testing Procedure

1. Start the Docker containers.
2. Verify that the application container is running.
3. Verify that the database container is running.
4. Open the application in the browser.
5. Test sensor-data submission.
6. Verify that data is stored.
7. Test AI analysis.
8. Verify alert generation.
9. Check application logs for errors.

Expected Result

The application should start successfully inside the Docker container and communicate with the database without dependency or environment-related errors.

11. Benefits of Git and Docker

Git Benefits

- Provides version control.
- Tracks changes in source code.
- Supports collaborative development.
- Allows branching and merging.
- Makes it possible to restore previous versions.
- Maintains a clear development history.

Docker Benefits

- Provides consistent environments.
- Simplifies deployment.
- Improves portability.

- Reduces dependency problems.
- Makes testing easier.
- Supports scalable deployment.

Combined Benefit

Git manages the **source code and development history**, while Docker manages the **application environment and deployment**.

Together, they improve the project's:

Collaboration → Version Control → Portability → Deployment → Maintainability

12. Challenges and Improvements

Challenges Encountered

1. Setting up the Git repository and branches.
2. Managing dependencies between application components.
3. Creating the Dockerfile correctly.
4. Configuring communication between application and database containers.
5. Handling port and dependency issues.
6. Testing the application inside the container.
7. Maintaining a clean commit history.

Improvements

The project can be improved by:

- Adding automated GitHub Actions for CI/CD.
- Using stronger authentication and authorization.
- Adding automated testing.
- Improving AI prediction accuracy.
- Adding real-time sensor integration.
- Using cloud deployment.
- Adding monitoring and logging tools.
- Implementing container orchestration using Kubernetes for large-scale deployment.

Future Enhancements

Future versions can include:

- Real-time IoT sensor integration.
- Advanced ML prediction models.
- Mobile application support.
- More detailed GIS visualization.
- Automatic mitigation recommendations.
- Cloud-based deployment.
- Real-time public noise maps.